

Function Calling / Tool Use

Letting the model reach beyond its own knowledge — tool calling, function calling, external APIs, the Model Context Protocol (MCP), tool selection, and handling tool results.

DIFFICULTY LEVEL

Intermediate

ESTIMATED READING TIME

55-70 minutes

ESTIMATED PRACTICE TIME

100-130 minutes

PREREQUISITES

Modules 01-05

LEARNING OUTCOMES

By the end of this module, you will be able to: explain how tool/function calling works end to end; define tools with clear schemas the model can select correctly; connect an LLM to external APIs safely; understand what MCP is and why it matters; reason about tool selection when multiple tools are available; and correctly handle and feed back tool results.

Table of Contents

1.	Introduction	
2.	Before You Begin	
3.	Main Lesson	
3.1	Tool Calling	
3.2	Function Calling	
3.3	External APIs	
3.4	MCP (Model Context Protocol)	
3.5	Tool Selection	
3.6	Tool Results	
4.	Visual Learning	
5.	Real World Applications	
6.	Practical Examples	
7.	Code Examples	
8.	Common Mistakes	
9.	Best Practices	
10.	Hands-on Activities	
11.	Mini Project	
12.	Review Questions	
13.	Cheat Sheet	
14.	Key Takeaways	
15.	Additional Resources	

1. Introduction

What is this topic?

Every module so far has treated the LLM as a closed box: text in, text out. This module opens that box up — letting the model call real functions, hit real APIs, and act on real, current data, instead of relying purely on what it memorized during training (Module 01, 3.13).

Why is it important?

Tool use is what turns an LLM from "a very good writer" into "a system that can actually do things" — check today's weather, query a database, run a calculation, book a meeting. It's the single biggest capability unlock covered in this course so far, and it's the foundation everything in Module 08 (AI Agents) is built on.



Why should AI Engineers learn it?

Almost every non-trivial AI product needs tool use: a support bot that checks a real order status, a coding assistant that runs real code, a research agent that runs real searches. Knowing how to define tools well, and how the model decides which to call, is core AI engineering skill.

Where is it used in the real world?

Claude Code running shell commands and editing files, customer support bots querying live order databases, and AI assistants booking calendar events all rely on tool calling under the hood.

Why is it relevant today?

MCP (Section 3.4), an open standard for connecting models to tools and data sources, has rapidly become a shared convention across the industry in 2025–2026 — understanding it now is directly relevant to how modern AI systems are actually built.

Note

This module builds directly on Module 04 (Structured Outputs) — tool calls are, under the hood, just structured, schema-validated output the API handles specially.

2. Before You Begin

RECAP

Typed outputs (Module 04, 3.8) — tool calling is the practical application of schema-constrained generation you already learned about.

RECAP

Model limitations (Module 01, 3.13) — tool use is precisely how you patch the gaps: math, live data, and actions the model can't do natively.



Warning

Tools that take real-world actions (sending emails, making purchases, deleting data) should never be executed blindly on model output. Always validate arguments and, for high-stakes actions, require explicit confirmation before executing.

3. Main Lesson

Each concept follows: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**.

✳️3.1 Tool Calling

DEFINITION

Tool calling is the general mechanism by which an LLM, instead of answering directly, outputs a structured request to invoke a specific tool (function) with specific arguments — which your code then executes.

SIMPLE EXPLANATION

You describe available tools to the model (name, description, expected arguments). When a user's request needs one, the model doesn't guess an answer — it returns a structured "call this tool with these arguments" response instead, and waits for the result.

💡 *Like a receptionist who doesn't personally know the answer to every question, but knows exactly which department to transfer the call to, and what information to give them.*

EVERYDAY EXAMPLE

Asked "What's the weather in Manila right now?", instead of guessing, the model returns a tool call: `get_weather(city="Manila")`, and your code actually fetches the real answer.

WHY IT MATTERS

Tool calling is the mechanism that connects an LLM's language understanding to real, current, or precise external capabilities it doesn't have on its own.

COMMON MISTAKES

Assuming the model executes the tool itself — it doesn't; your application code always does the actual execution.

BEST PRACTICES

Keep the loop explicit in your mental model: model requests a call → your code executes it → you feed the result back → model continues (Section 3.6).

SUMMARY

Tool calling is the structured request-response mechanism that lets a model ask for external help instead of guessing.

⚙️3.2 Function Calling

DEFINITION

Function calling is essentially the same concept as tool calling — the term commonly used when the "tool" is a specific function in your own codebase, with a defined name, description, and parameter schema.

SIMPLE EXPLANATION

You describe your function like a mini API spec: what it's called, what it does, and what arguments (with types) it expects. The model uses this spec to decide when and how to "call" it — matching the output format from Module 04 (Pydantic models, typed outputs) almost exactly.

```
Function spec:
name: get_order_status
description: "Look up the status of a customer order"
parameters:
  order_id: string (required)
```

💡 *Like a well-documented API endpoint — the model reads the "API docs" (your function schema) and knows exactly*

how to request it correctly.

EVERYDAY EXAMPLE

Defining `calculate_shipping(weight_kg, destination_country)` so the model can request an exact shipping cost instead of guessing one from training data.

WHY IT MATTERS

Precise function schemas are what makes tool calling reliable — vague or missing descriptions lead to the model calling the wrong function or with wrong arguments.

COMMON MISTAKES

Writing vague function descriptions ("does stuff with orders") that don't give the model enough signal to know when to use it.

BEST PRACTICES

Write function descriptions the way you'd document a public API for another developer — specific, with examples of when to use it.

SUMMARY

Function calling is tool calling applied to your own codebase's functions — reliability lives and dies by how clearly each function is described.

🌐 3.3 External APIs

DEFINITION

External APIs are third-party services (weather data, payment processing, search engines, databases) that a tool can call out to, extending what the model can access far beyond your own codebase.

SIMPLE EXPLANATION

A tool's implementation doesn't have to be your own logic — it's often a thin wrapper that calls an external API and returns the result back to the model in the tool-result format.

💡 *Like a personal assistant who doesn't personally know today's stock prices, but knows exactly which phone number to call to get the real, current number instantly.*

EVERYDAY EXAMPLE

A `get_stock_price(ticker)` tool that, under the hood, calls a financial data API and returns the current price to the model.

WHY IT MATTERS

External APIs are how AI systems get truly live, current, or specialized data (stock prices, weather, flight status) that no LLM could have memorized accurately, since it constantly changes.

COMMON MISTAKES

Not handling external API failures (timeouts, rate limits, bad responses) gracefully, causing the whole tool-use flow to break.

BEST PRACTICES

Wrap external API calls with error handling and return a clear, structured error to the model if the call fails, so it can respond sensibly instead of crashing.

SUMMARY

External APIs extend tool use beyond your own code to any live, specialized, or third-party data source.

🔗 3.4 MCP (Model Context Protocol)

DEFINITION

MCP (Model Context Protocol) is an open standard that defines a common way for AI applications to connect to tools, data sources, and external systems — so a tool built once can work across many different AI applications without custom integration each time.

SIMPLE EXPLANATION

Before MCP, connecting an LLM app to, say, Google Drive or Slack meant custom integration code for every single AI product. MCP standardizes that connection — an "MCP server" for Slack can be plugged into any MCP-compatible AI application, the same way a USB device works across different computers without custom drivers for each one.



💡 Like USB-C standardizing device connections — before it, every device needed its own proprietary cable; now one standard connector works across many devices and manufacturers.

EVERYDAY EXAMPLE

Claude connecting to a Google Drive MCP server can list, search, and read files — and the same MCP server could be plugged into a different MCP-compatible AI product with no changes.

WHY IT MATTERS

MCP dramatically reduces the "N tools × M AI apps = N×M custom integrations" problem down to "N tools + M apps," making the whole ecosystem of AI-connected tools much easier to build and maintain.

COMMON MISTAKES

Confusing MCP (a connection/integration standard) with tool calling itself (the underlying mechanism) — MCP is a standardized way to expose tools, not a replacement for the concept of tool calling.

BEST PRACTICES

When integrating with common external systems (Slack, GitHub, Google Drive, databases), check first whether an existing MCP server already covers it before building custom integration code.

SUMMARY

MCP is a standard protocol for connecting AI applications to tools and data sources — the "USB-C for AI tools" that avoids one-off integrations per app.

🔗 3.5 Tool Selection

DEFINITION

Tool selection is the model's process of deciding, given a set of available tools, which (if any) to call for a given request.

SIMPLE EXPLANATION

When multiple tools are available, the model reads each tool's name and description and matches them against the user's request — good descriptions lead to good selection; ambiguous or overlapping descriptions lead to wrong or inconsistent tool choices.

💡 Like a triage nurse deciding which specialist to refer a patient to — clear specialties (a cardiologist vs. a dermatologist) make routing easy; overlapping vague specialties make it error-prone.

EVERYDAY EXAMPLE

Given both `search_web(query)` and `search_internal_docs(query)`, the model needs clear enough descriptions to know that a question about "our company's leave policy" should call the internal docs tool, not the web search.

WHY IT MATTERS

As the number of available tools grows (especially in agent systems, Module 08), tool selection accuracy becomes a major reliability factor — the model can only use a tool correctly if it picks the right one in the first place.

COMMON MISTAKES

Providing many tools with overlapping or vague purposes, causing the model to pick the wrong one or hesitate between options.

BEST PRACTICES

Keep tool purposes distinct and clearly described; avoid offering many tools that do nearly the same thing; test selection accuracy across realistic queries.

SUMMARY

Tool selection is only as reliable as the clarity and distinctiveness of your tool descriptions — ambiguity here causes real, hard-to-debug failures.

3.6 Tool Results

DEFINITION

A tool result is the output your code sends back to the model after executing a requested tool call, which the model then uses to continue generating its final response.

SIMPLE EXPLANATION

Tool calling is a full round trip: the model requests a call → your code executes it → your code sends the result back into the conversation → the model reads that result and produces its actual answer to the user.

Model: Tool Call



Your Code: Execute



Tool Result



Model: Final Answer

 Like a research assistant who runs off to check a fact, comes back and reports "the answer is X," and only then does the person writing the final report incorporate that fact into their conclusion.

EVERYDAY EXAMPLE

After calling `get_weather(city="Manila")` and receiving `{"temp_c": 31, "condition": "sunny"}` as the result, the model can now say "It's sunny and 31°C in Manila right now."

WHY IT MATTERS

How you format and return tool results directly affects the quality of the model's final answer — a raw, messy API response is harder for the model to use well than a clean, relevant result.

COMMON MISTAKES

Returning an entire raw API response (with lots of irrelevant fields) instead of just what's needed. Not clearly signaling an error result vs. a successful one.

BEST PRACTICES

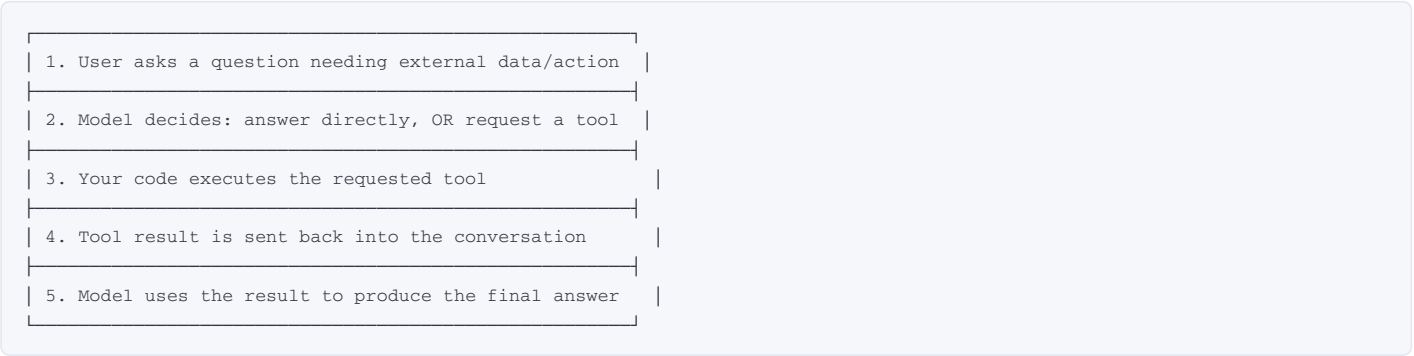
Return concise, relevant tool results; clearly distinguish success from error results so the model can respond appropriately in either case.

SUMMARY

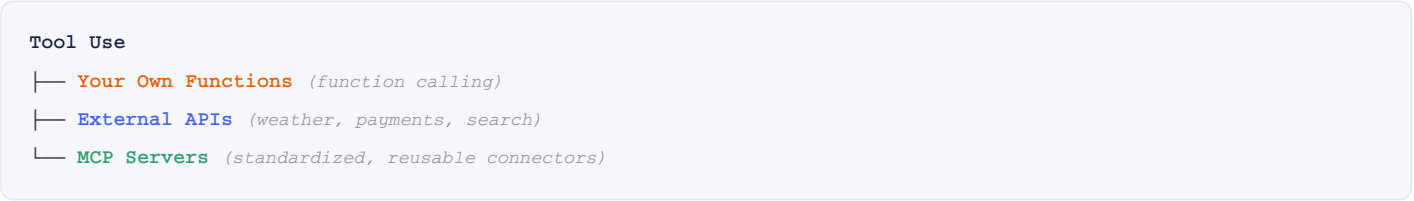
Tool results close the loop — clean, relevant results lead directly to better final answers from the model.

4. Visual Learning

🔄The Full Tool Use Loop



🌳Tool Ecosystem



📊Good vs. Poor Tool Descriptions

Poor Description	Good Description
"Does order stuff"	"Look up the current shipping status of a customer order by order ID"
"Search"	"Search the internal company knowledge base for policy documents"
"Math"	"Evaluate a precise arithmetic expression and return the exact numeric result"

5. Real World Applications

Product	How these fundamentals show up
Claude Code / Cursor	Function calling drives running shell commands, editing files, and searching a codebase.
Claude with connectors (Google Drive, Slack, etc.)	MCP servers expose these services as standardized tools Claude can call.
Customer support bots	Function calling queries live order/account databases via internal APIs instead of guessing.
Research/finance assistants	External API tools fetch live stock prices, news, or search results.

6. Practical Examples

Beginner: Spotting a tool-use need

List 5 questions you might ask an AI assistant that it truly cannot answer from training data alone (e.g., "What's my account balance right now?"). These are all tool-use candidates.

Beginner: Writing a tool description

Write a tool schema (name, description, parameters) for a fictional `get_traffic(origin, destination)` tool, following the "good description" pattern from Section 4.

Intermediate: Full round trip by hand

On paper, trace through the 5-step loop (Section 4) for a real example: "What's 847×293 ?" — write out what the model's tool call would look like, what your code would return, and what the model's final answer would be.

Advanced: Tool selection ambiguity

Design 2 tools with deliberately overlapping purposes (e.g., `search_web` and `search_news`), then write 3 test queries and predict which tool the model should pick for each. Test your predictions with a real API call if possible.

7. Code Examples

🐍Python — Defining and handling a tool call

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

# 1. Define the tool schema (like a function's public API)
tools = [{
    "name": "get_weather",
    "description": "Get the current weather for a specific city.",
    "input_schema": {
        "type": "object",
        "properties": {
            "city": {"type": "string", "description": "City name, e.g. Manila"}
        },
        "required": ["city"]
    }
}]

def get_weather(city):
    # In real code, this would call a real weather API
    return {"city": city, "temp_c": 31, "condition": "sunny"}

# 2. Send the user message with tools available
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    tools=tools,
    messages=[{"role": "user", "content": "What's the weather in Manila?"}]
)

# 3. Check if the model requested a tool call
for block in response.content:
    if block.type == "tool_use" and block.name == "get_weather":
        result = get_weather(**block.input) # execute it

# 4. Send the tool result back so the model can finish answering
follow_up = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    tools=tools,
    messages=[
        {"role": "user", "content": "What's the weather in Manila?"},
        {"role": "assistant", "content": response.content},
        {"role": "user", "content": [{
            "type": "tool_result",
            "tool_use_id": block.id,
            "content": str(result)
        }]}
    ]
)
print(follow_up.content[0].text)
```

🌐JavaScript — Same pattern, Node.js

```
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";
const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const tools = [{
  name: "get_weather",
  description: "Get the current weather for a specific city.",
  input_schema: {
    type: "object",
    properties: { city: { type: "string" } },
    required: ["city"],
  },
}];

function getWeather({ city }) {
  return { city, tempC: 31, condition: "sunny" }; // mock; call a real API here
```

```

}

const response = await client.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 300,
  tools,
  messages: [{ role: "user", content: "What's the weather in Manila?" }],
});

const toolBlock = response.content.find((b) => b.type === "tool_use");
if (toolBlock) {
  const result = getWeather(toolBlock.input);
  const followUp = await client.messages.create({
    model: "claude-sonnet-4-6",
    max_tokens: 300,
    tools,
    messages: [
      { role: "user", content: "What's the weather in Manila?" },
      { role: "assistant", content: response.content },
      { role: "user", content: [{ type: "tool_result", tool_use_id: toolBlock.id, content: JSON.stringify(result) }] },
    ],
  });
  console.log(followUp.content[0].text);
}

```



Warning

For tools that take real-world action (sending money, deleting records, sending messages), add an explicit confirmation step before execution — never auto-execute high-stakes tool calls straight from model output.

8. Common Mistakes

- **Writing vague tool descriptions**, causing poor tool selection accuracy.
- **Assuming the model executes tools itself** — your application code always performs the actual execution.
- **Not handling external API failures**, breaking the whole tool-use flow on any hiccup.
- **Offering many overlapping tools**, confusing tool selection.
- **Returning bloated, irrelevant raw API responses** as tool results instead of clean, focused data.
- **Auto-executing high-stakes actions** without validation or confirmation.

9. Best Practices

- **Write tool descriptions like public API documentation** — specific, with a clear "when to use this" signal.
- **Keep tool purposes distinct** — avoid multiple tools that do nearly the same thing.
- **Handle external API errors gracefully** and return a clear error result the model can react to sensibly.
- **Return concise, relevant tool results**, not entire raw API payloads.
- **Gate high-stakes actions behind explicit confirmation** before real-world execution.
- **Check for an existing MCP server** before building custom integrations to common external systems.

10. Hands-on Activities

🟡 Easy 5 exercises

1. List 5 real questions an AI assistant needs a tool to answer correctly, and explain why for each.
2. Write a tool schema (name, description, parameters) for a fictional `convert_currency(amount, from_currency, to_currency)` tool.
3. Compare a vague tool description vs. a specific one for the same tool, and predict which leads to more reliable selection.
4. Diagram (on paper) the 5-step tool use loop for a "book a restaurant reservation" tool.
5. Research one existing MCP server (e.g., for GitHub or Slack) and summarize what it lets an AI app do.

🟠 Intermediate 3 exercises

1. Implement the weather tool example from Section 7 end to end and test it with 3 different city names.
2. Add a second, distinct tool (e.g., a currency converter) and test that the model selects the correct one across mixed queries.
3. Deliberately make your tool's implementation throw an error, and design a clean error-result format the model can react to gracefully.

🔴 Challenge 2 exercises

1. Build a 3-tool system (e.g., weather, currency conversion, and a simple calculator) and test tool selection accuracy across 10 varied queries, including some ambiguous ones.
2. Design (on paper) a confirmation flow for a hypothetical "send payment" tool — what gets shown to the user before execution, and what happens if they decline.

11. Mini Project

Goal

Build a small multi-tool assistant that can answer questions requiring either a weather lookup or a basic calculation, correctly selecting the right tool each time.

Requirements

- An Anthropic API key
- Python or JavaScript with the Anthropic SDK

Step-by-Step Instructions

1. Define two tools: `get_weather(city)` (mocked data is fine) and `calculate(expression)` (use a safe eval or math library).
2. Implement both functions in your code.
3. Send a user message, check for a `tool_use` block, execute the matching function, and send the result back.
4. Test with at least 5 different questions: some needing weather, some needing calculation, and one needing neither (should get a direct answer, no tool call).
5. Print which tool (if any) was selected for each question, and the final answer.

Expected Output

Q: "What's the weather in Cebu?"	→ Tool: <code>get_weather</code>	→ Answer: ...
Q: "What's 482 * 17?"	→ Tool: <code>calculate</code>	→ Answer: 8194
Q: "What's the capital of Japan?"	→ Tool: <code>none</code>	→ Answer: Tokyo

Possible Improvements

- Replace the mocked weather function with a real weather API call.
- Add a confirmation step for the calculate tool if the expression looks unusually complex.
- Look into your provider's MCP support and see if an existing MCP server could replace one of your custom tools.

12. Review Questions

Multiple Choice (10)

1. Tool calling means the model:

- A) Executes the tool itself B) Outputs a structured request to invoke a tool, which your code executes C) Ignores available tools D) Always answers directly

2. A good function description should be written like:

- A) A vague hint B) Clear, specific public API documentation C) A single word D) An error message

3. External APIs in tool use provide:

- A) Static training data B) Live, current, or specialized data beyond the model's training C) Nothing new D) A replacement for prompting

4. MCP (Model Context Protocol) is best described as:

- A) A single company's private API B) An open standard for connecting AI apps to tools/data sources C) A prompting technique D) A type of temperature setting

5. Tool selection accuracy depends heavily on:

- A) Model temperature only B) Clear, distinct tool descriptions C) The color of the UI D) Max tokens

6. A tool result is:

- A) The user's original question B) The output sent back to the model after executing a tool call C) A type of temperature D) The system prompt

7. Why should high-stakes tool actions require confirmation?

- A) It's always required by every API B) To prevent unintended real-world actions from being auto-executed C) It lowers cost D) It increases temperature

8. Overlapping, vague tools tend to cause:

- A) Better performance B) Poor or inconsistent tool selection C) Lower token cost D) No effect at all

9. What should you do if an external API call inside a tool fails?

- A) Crash the app silently B) Return a clear error result the model can react to C) Ignore it and return nothing D) Always retry infinitely with no limit

10. MCP most directly solves which problem?

- A) Model hallucination B) Needing custom integration code for every tool-app pairing C) Token pricing D) Context window size

Identification (5)

1. _____ is the structured mechanism by which a model requests an external tool be invoked.

2. _____ refers to defining and exposing your own codebase's functions for the model to call.

3. _____ is an open standard for connecting AI applications to tools and data sources.

4. _____ is the model's process of deciding which available tool (if any) to call.

5. _____ is the data sent back to the model after a tool call has been executed.

True or False (5)

1. The model itself executes tool calls directly, without any application code involved.

2. MCP allows a tool built once to work across multiple different AI applications without custom integration each time.

3. Vague tool descriptions generally improve tool selection accuracy.

4. Tool results should ideally be concise and relevant, not entire raw API payloads.

5. High-stakes tool actions should generally require confirmation before execution.

Situational (5)

Situational (5)

- 1. Your assistant keeps calling the wrong tool between two similar-sounding options. What's the most likely root cause and fix?**
- 2. A user asks for a live currency conversion rate, but your model tries to answer directly with a possibly outdated number. What's missing from your setup?**
- 3. Your weather API occasionally times out, and your app crashes when that happens. What should you change?**
- 4. You want to integrate with Slack without writing a custom integration from scratch. What should you check for first?**
- 5. You're building a "delete customer record" tool. What extra safeguard should you add before it executes?**

Answer Key

Multiple Choice: 1-B, 2-B, 3-B, 4-B, 5-B, 6-B, 7-B, 8-B, 9-B, 10-B

Identification: 1-Tool calling, 2-Function calling, 3-MCP (Model Context Protocol), 4-Tool selection, 5-Tool result

True or False: 1-False, 2-True, 3-False, 4-True, 5-True

Situational (sample reasoning):

1. Overlapping/vague tool descriptions — rewrite them to be more specific and distinct.
2. A currency-conversion tool connected to a live external API, so the model calls it instead of guessing.
3. Add error handling around the API call and return a clear error result instead of letting it crash.
4. Check whether an existing MCP server for Slack already exists before building custom integration code.
5. An explicit confirmation step before the delete action actually executes.

13. Cheat Sheet

🔄 The Loop

Model requests tool → your code executes → result sent back → model gives final answer.

✳️ Good Tool Descriptions

Write like public API docs: specific purpose, clear "when to use," well-typed parameters.

🔌 MCP

Open standard connecting AI apps to tools/data — build once, use across many apps.

⚠️ Safety

Validate arguments; handle API errors gracefully; confirm before high-stakes actions.

14. Key Takeaways

- Tool calling lets a model request external help instead of guessing — your application code always does the actual execution.
- Function calling and tool calling are the same mechanism, typically distinguished by whether the "tool" is your own code or an external service.
- External APIs give the model access to live, current, or specialized data it could never have memorized accurately.
- MCP standardizes connections between AI apps and tools/data sources, avoiding one-off integrations per app.
- Tool selection accuracy depends heavily on clear, distinct tool descriptions.
- Clean, relevant tool results lead to better final answers; high-stakes actions need confirmation before execution.

15. Additional Resources

Official Documentation

- Anthropic — Tool use overview: docs.claude.com
- Model Context Protocol — official specification and docs: modelcontextprotocol.io

Related Modules

- Module 04 — Structured Outputs (the schema mechanics underlying tool calls)
- Module 08 — AI Agents (tool use as the action mechanism inside agent loops)
- Module 10 — AI Safety (validating tool inputs/outputs safely)

Communities & Further Learning

- MCP server directories/registries for discovering existing connectors before building custom ones.